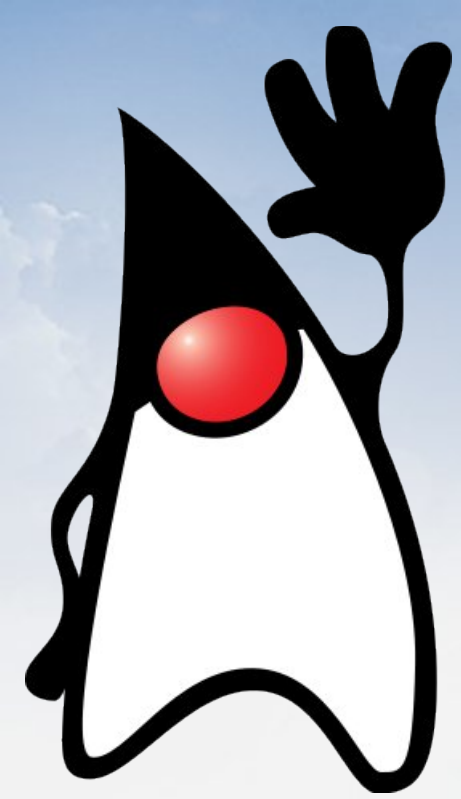




Queues in Java

COP 3330
Department of Computer Science
University of Central Florida
Dr. Bacanli

What is a Queue?

- A queue is a sequence of nodes in which we can only access the first item. We add to the last item
- Ex: People in the free pizza line

34	3	46	7
----	---	----	---

Queue uses FIFO.

Removing from queue is called dequeue

Adding to queue is called enqueue

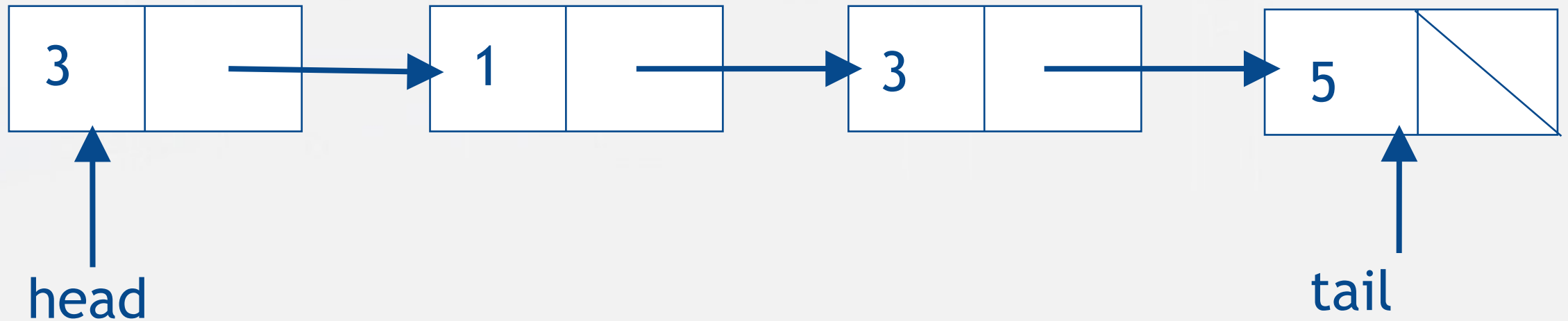
Getting the first element without removal is called peek

Implementation

- Enqueue, dequeue and peek, all have $O(1)$ run time complexity
 - There is no traversal of elements in queue, as a standard operation
- Queue can be implemented with arrays and Linked Lists.
- What are the advantages and disadvantages of those implementations?
- Queue is used in
 - Networking (serving packets)
 - Printers getting print orders
 - cases where first element/case has priority among others

Queue as a linked list

In a singly linked list we store the elements as shown below in a queue. We need to store the tail and head. We remove from head. Add to the tail. What if we don't store the tail?



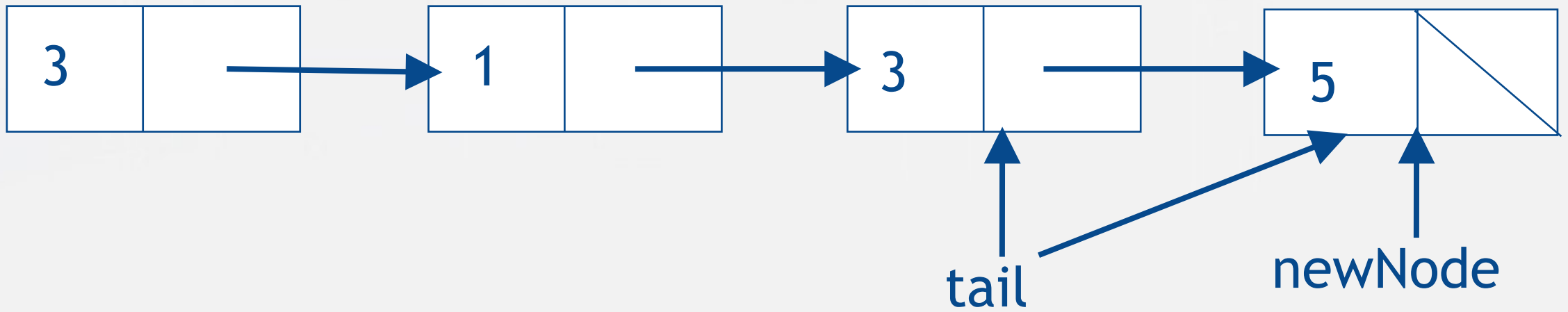
Enqueue to Queue

```
Node newNode=new Node(3);
```

```
tail.setNext(newNode);
```

```
tail = newNode;
```

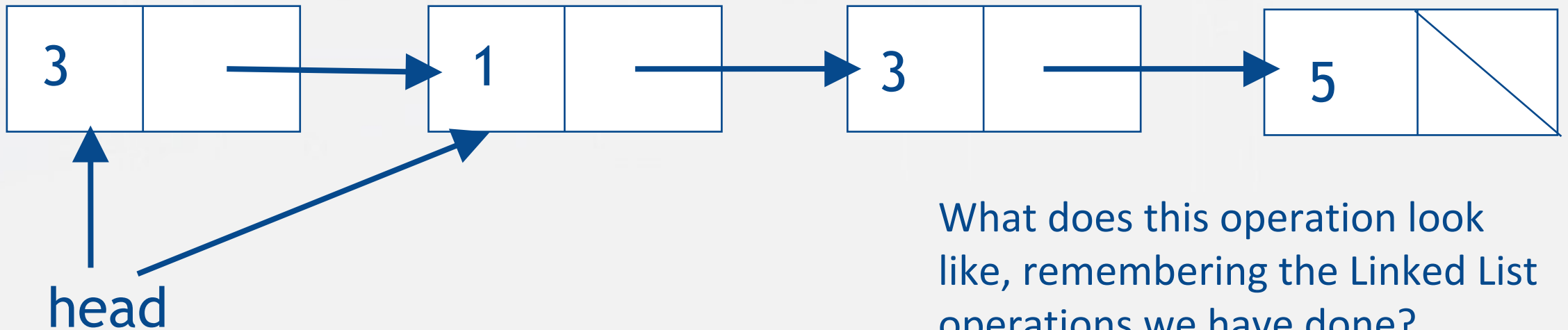
What does this operation look like, remembering the Linked List operations we have done?



Dequeue from Queue

```
returnedElem = head.getItem(); // store the item somewhere
```

```
head = head.getNext(); // make head node unreachable
```



What does this operation look like, remembering the Linked List operations we have done?

Enqueue to Queue

```
tail = 4;
```

```
queueArr[tail+1] = 9;
```

```
tail = tail+1;
```

What happens if we filled all the positions of our starting array (queueArr)?

1	4	7	3	2	9
---	---	---	---	---	---

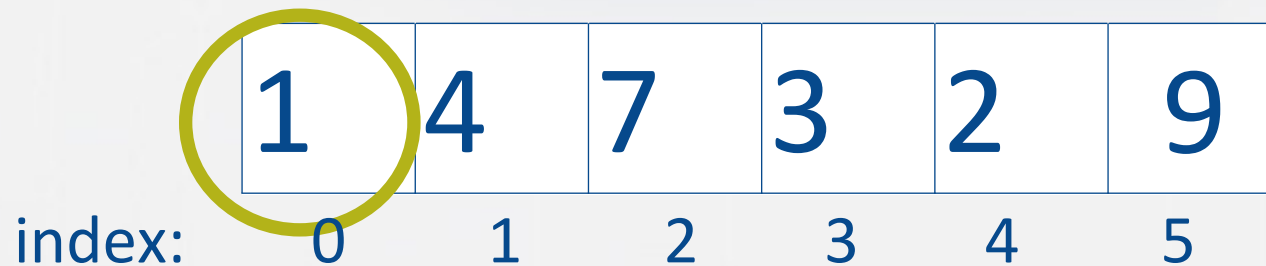
index: 0 1 2 3 4 5

How can we get away from this problem? Will runtime change?

Dequeue from Queue

```
head = 0;  
returnedElem = queueArr[head];  
head = head + 1;
```

We need to check if the queue is empty!
head and tail always increase!



What happens to
1 when we
dequeue?

```
public int peek(){  
    return queueArr[head];  
}
```